

Sistemas de Gestión de Seguridad

Grado en Ingeniería Informática de Gestión y sistemas
de información

Sistema Web: Auditoría

2025-2026

Fecha:

Bilbao, a 7 de noviembre de 2025

Grupo:

Podcast

Autores:

Fernández Pajares, María

Fernández Molano, Iker

Galván Carrillo, Nahia

Molinero Medel, Lucía

Rivera Urretxi, Aitzol

Tapias Monasterio, Paula

ÍNDICE

1. ROTURA DE CONTROL DE ACCESO.....	2
1.1. CROSS-SITE REQUEST FORGERY (CSRF).....	2
1.2. ACCESO Y PERMISO DE MODIFICACIÓN DE DATOS DE OTROS USUARIOS.....	2
2. FALLOS CRIPTOGRÁFICOS.....	3
2.1. CONTRASEÑA SIN PBKDFs Y TEXTO PLANO.....	3
2.2. CONEXIONES INSEGURAS CON LA BASE DE DATOS.....	3
3. INYECCIÓN.....	4
3.1. INYECCIÓN SQL.....	4
4. DISEÑO INSEGURO.....	5
4.1 FALTA DE MEDIDAS DE PROTECCIÓN CONTRA LOS BOTS.....	5
4.2 AUSENCIA DE LÍMITES Y CONTROLES SOBRE EL USUARIO.....	5
4.3 VALIDACIONES DE DATOS SÓLO EN EL LADO DEL CLIENTE.....	5
4.4 MEDIDAS COMO TOKENS CSRF, RATE-LIMITING O VERIFICACIÓN POR ROLES.....	6
4.5 ESTRATEGIA DE GESTIÓN DE ERRORES.....	6
5. CONFIGURACIÓN DE SEGURIDAD INSUFICIENTE.....	7
5.1. COOKIES DE SESIÓN SIN FLAGS SEGUROS Y SIN REGENERACIÓN DE ID.....	7
5.2. SERVIDOR SOBRE HTTP (SIN TLS/HTTPS) Y SIN HTTPS.....	8
5.3. CABECERAS DE SEGURIDAD HTTP AUSENTES (CSP, X-Frame-Options, X-Content-Type-Options, REFERRER-POLICY).....	8
6. COMPONENTES VULNERABLES Y OBSOLETOS.....	10
6.1. DESCONOCIMIENTO DE VERSIONES EXACTAS.....	10
6.2. USO DE SOFTWARE OBSOLETO O SIN SOPORTE.....	10
6.3. DEPENDENCIAS EXTERNAS NO CONTROLADAS.....	10
7. FALLOS DE IDENTIFICACIÓN Y AUTENTICACIÓN.....	12
7.1. AUSENCIA DE AUTENTICACIÓN.....	12
7.2. FALTA DE LIMITACIÓN DE INTENTOS Y MENSAJES.....	12
8. FALLOS EN LA INTEGRIDAD DE DATOS Y SOFTWARE.....	13
8.1. INCORPORACIÓN DE RECURSOS EXTERNOS SIN VERIFICAR SU INTEGRIDAD.....	13
8.2. FALTA DE VERIFICACIÓN DE INTEGRIDAD DEL SOFTWARE LOCAL.....	13
9. FALLOS EN LA MONITORIZACIÓN DE LA SEGURIDAD.....	14
10. BIBLIOGRAFÍA.....	15

1. ROTURA DE CONTROL DE ACCESO

1.1. CROSS-SITE REQUEST FORGERY (CSRF)

En la auditoría anterior se identificó que el sistema web presentaba debilidades ante acciones no autorizadas aprovechando la sesión activa.

Con el fin de solucionarlo, al iniciar una sesión generamos un token CSRF aleatorio seguro, el cual se guarda como atributo de la sesión, de la cookie, en caso de no estar creado. Este token se guarda en todos los formularios de tipo POST como campo oculto.

De esta manera, cuando se recibe la petición POST se comparan el token CSRF guardado en la sesión y el recibido por el formulario. Si coinciden, se continúa con el proceso normalmente; Si no coinciden o no existe, el proceso deniega el acceso y se bloquea.

Todo lo explicado anteriormente queda configurado en el archivo db_connect.php, el cual se incluye en todos los archivos php que hacen uso de la base de datos.

1.2. ACCESO Y PERMISO DE MODIFICACIÓN DE DATOS DE OTROS USUARIOS

Este tipo de vulnerabilidad surgía al poder manipular el parámetro “user” de la URL de show/modify-user. En esta nueva versión se han tomado varias medidas, que denominaremos “muros”.

El primer muro se basa en comparar el usuario guardado como atributo en la cookie de sesión con el valor del parámetro user de la URL; si no coinciden se bloquea el acceso y no se permite acceder a los datos del otro usuario, si coinciden, se continúa con normalidad.

El segundo muro consiste en que, a diferencia de la anterior auditoría, todas las consultas que se realizan utilizan el valor del atributo de usuario de la sesión. Es decir, aunque el URL cambie, los datos siempre serán aquellos asociados al usuario de la sesión.

2. FALLOS CRIPTOGRÁFICOS

2.1. CONTRASEÑA SIN *PBKDFs* Y TEXTO PLANO

En la versión anterior, se identificó que la contraseña era almacenada en texto plano, de manera literal. Como solución se ha implementado un algoritmo de hashing, en este caso, Bcrypt.

Por un lado, este algoritmo además de realizar varias iteraciones de hash a la contraseña, añade un salt único. Se almacena de forma segura. Esto dificulta, por tanto, los ataques por diccionario o por fuerza bruta.

Por otro lado, se utiliza otra función (`password_verify()`) segura, ya que no expone la contraseña al texto plano, para poder verificar que la contraseña dada, por ejemplo, al iniciar sesión en texto plano, es la correcta.

2.2. CONEXIONES INSEGURAS CON LA BASE DE DATOS

Las conexiones con la base de datos no tenían ningún protocolo de cifrado configurado, lo que exponía las acciones, pues no estaban cifradas.

Por tanto, para solucionarlo, hemos configurado que las conexiones con la base de datos se realicen con el protocolo TLS/SSL, para garantizar la transmisión de información segura entre cliente y servidor. Para ello, mediante el servicio `certgen` definido en `docker-compose` generamos los certificados necesarios:

- El certificado de autoridad, sirve para indicar la fiabilidad y legitimidad del servidor .
- La clave del servidor, para descifrar archivos cifrados con el certificado del servidor y firmar mensajes.
- El certificado del servidor, para que el cliente verifique la identidad del servidor con el certificado de autoridad y, como dispone de la clave pública del servidor, es usado para cifrar la clave de sesión.

3. INYECCIÓN

3.1. INYECCIÓN SQL

Como ya se indicó en la anterior auditoría, esta fue una vulnerabilidad que ya estaba solventada. Las medidas que se tomaron para evitar los ataques por inyección SQL fueron:

1. Se prepara la sentencia SQL, indicando con “?” el lugar donde deben ir los parámetros solicitados. De esta manera, se tomará de manera literal el valor dado, no se podrá interpretar como parte de la consulta.

```
$stmt = $conexion->prepare("INSERT INTO item (nombre, anio, combustible, caballos, precio) VALUES (?, ?, ?, ?, ?)");
```

2. Se asocian los valores, conocido su tipo de dato (s=string, i=integer, d=double), a los parámetros previamente indicados en la preparación. Esto asegura el correcto manejo de los parámetros.

```
$stmt->bind_param("sisid", $nombre, $anio, $combustible, $caballos, $precio);
```

3. Por último, se ejecuta la sentencia SQL.

```
$stmt->execute();
```

4. DISEÑO INSEGURO

4.1 FALTA DE MEDIDAS DE PROTECCIÓN CONTRA LOS BOTS

Para solucionar este problema, hemos añadido un contador en Iniciar Sesión. Es decir, una vez te hayas registrado, solo tienes 3 oportunidades para iniciar sesión bien. En el caso de que falles 3 veces seguidas, se bloqueará durante 3 minutos la sesión, y una vez pasados ya podrás volver a intentar.

Además para proteger nuestro sistema ante los bots y las ralentizaciones en el sistema, hemos añadido un rate-limiting, es decir desde la misma IP sólo te puedes registrar 3 veces cada 30 segundos. De este mismo modo, hemos añadido este sistema para add item y delete item para comprobar que desde la misma IP hagan como máximo 10 acciones cada minuto. Esto lo hacemos ya que un bot podría enviar miles de peticiones seguidas para crear o borrar cosas o incluso borrar todos los ítems del sistema.

4.2 AUSENCIA DE LÍMITES Y CONTROLES SOBRE EL USUARIO

Para solucionar la ausencia de límites, hemos editado nuestra base de datos, además de las validaciones que ya teníamos, hemos añadido nuevas. El nombre de usuario tiene 30 caracteres como máximo. El nombre 50, los apellidos y el correo 100. Además la contraseña tiene que tener como mínimo 8 caracteres.

Nuestro sistema web cuenta con validación de campos vacíos, de longitudes máximas, de formatos (DNI valido, correo, teléfono..) Además hemos añadido una nueva validación que consiste en que no se puede poner como fecha de nacimiento una fecha futura.

Estas correcciones se han añadido al formulario de registro y al de modificar los datos.

4.3 VALIDACIONES DE DATOS SÓLO EN EL LADO DEL CLIENTE

Nuestras validaciones, ya mencionadas anteriormente, no se realizan únicamente en el lado del cliente. Aunque utilicemos el Java Script para comprobar el formato correcto, por ejemplo, el DNI, también se encuentran en el lado del servidor, en el php.

Con esto, conseguimos que aunque el usuario desactive el Javascript o intente manipular los datos enviados al servidor, el sistema los vuelve a validar por segunda vez garantizando la seguridad de estos campos.

4.4 MEDIDAS COMO TOKENS CSRF, RATE-LIMITING O VERIFICACIÓN POR ROLES

Como hemos mencionado anteriormente, se han implementado medidas contra el ataque, hemos insertado tokens csrf, hemos añadido rate-limiting. En cuanto a los roles, hemos considerado que todo el mundo puede realizar las mismas funciones dentro del sistema web (no existe un administrador). Por lo que no consideramos que esta sea necesaria.

4.5 ESTRATEGIA DE GESTIÓN DE ERRORES

Para evitar que los errores se muestren en pantalla e indiquen datos comprometidos, hemos redirigido los errores a una carpeta (logs) donde se almacenan los errores que saltan en nuestro servidor web. Los errores se deberían redirigir a una carpeta privada de logs dentro del repositorio, pero si no consigue escribirlo, se manda a la carpeta por defecto del ordenador en la que sí puede escribirse, es decir, /tmp.

5. CONFIGURACIÓN DE SEGURIDAD INSUFICIENTE

5.1. COOKIES DE SESIÓN SIN FLAGS SEGUROS Y SIN REGENERACIÓN DE ID

En la versión original, las páginas del sitio web que manejaban sesiones (login.php, items.php, show_user.php, etc.) usaban `session_start()` sin configurar bien la seguridad de las cookies de sesión.

Todo eso implicaba que las cookies:

- ➡ Podían viajar por conexiones sin cifrar (secure estaba desactivado).
- ➡ Se podían leer con JavaScript (httponly tampoco estaba activado).
- ➡ No tenían un contexto de seguridad definido (samesite sin valor).

Además, al iniciar sesión, el ID de la sesión no se renovaba. Esto abría la puerta a ataques de session fixation, donde algún atacante podría obligar a usar un ID de sesión que él controlaba.

Para solucionar estos fallos, creamos un archivo `config.php` para manejar toda la configuración de seguridad de las sesiones y las cabeceras en un solo lugar. En él, añadimos los parámetros correctos para hacer más seguras las cookies y las sesiones de PHP:

```
session_set_cookie_params([
    'lifetime' => 0,           → La cookie expira al cerrar el navegador
    'path' => '/',
    'domain' => '',
    'secure' => isset($_SERVER['HTTPS']), → Solo se envía por HTTPS
    'httponly' => true,        → Evita que JavaScript acceda a la cookie
    'samesite' => 'Lax'       → Impide envío de cookies entre sitios no confiables
]);
ini_set('session.use_strict_mode', 1); → Impide el uso de IDs de sesión no válidos
session_start();
```

Además, en el archivo `login.php`, después de confirmar que el usuario introdujo de manera correcta sus credenciales, añadimos esto:

```
session_regenerate_id(true);
```

Esto obliga a crear un nuevo ID para la sesión y borra el anterior. Así, si alguien supiera un ID viejo, no podría usarlo después de que el usuario se conecte. Con esto, las sesiones del usuario están aisladas, seguras y a salvo de que roben la sesión.

5.2. SERVIDOR SOBRE HTTP (SIN TLS/HTTPS) Y SIN HTTPS

En la auditoría anterior, se ha visto que la página web mandaba información sin cifrar por HTTP. Esto dejaba al descubierto nombres de usuario, claves, datos de formularios y cookies, fáciles de robar con ataques de tipo Man-in-the-Middle (MITM).

Para arreglarlo, hemos configurado el servidor Apache para que funcione con HTTPS activando TLS.

El servicio certgen definido en docker-compose crea automáticamente los certificados que se necesitan (server.crt y server.key). Los genera de forma segura, como certificados autofirmados, y los guarda en la carpeta ssl/. De forma que cada usuario tenga su clave y certificado propio.

El puerto que usa el servidor web ha sido modificado del puerto 80 al 443, de HTTP a HTTPS. De esta manera, sólo podrá accederse al servidor por medio de conexiones seguras.

Con todos estos cambios, el sistema ahora asegura que los datos estén a salvo y que nadie pueda verlos ni modificarlos, acabando con los problemas de falta de cifrado de HTTP.

5.3. CABECERAS DE SEGURIDAD HTTP AUSENTES (CSP, X-Frame-Options, X-Content-Type-Options, REFERRER-POLICY)

El sitio web no mandaba las cabeceras HTTP de seguridad, lo que dejaba la página web vulnerable a varios ataques:

- ➡ Clickjacking: meter la página en un iframe de otro dominio para engañar al usuario.
- ➡ MIME sniffing: obligar a que el navegador interprete mal los archivos (por ejemplo, un .txt como si fuera .js).
- ➡ Fugas de datos: a través de la cabecera Referer.
- ➡ Cross-Site Scripting (XSS): porque se podían ejecutar scripts sin control.

Sin el uso de estas cabeceras, el navegador no pone las restricciones de seguridad que son necesarias. Así que se han agregado las cabeceras que faltaban en el archivo config.php, que ahora se usa al principio de todas las páginas PHP de la web:

header("X-Frame-Options: SAMEORIGIN"); → Evita que la web se meta en iframes de fuera

header("X-Content-Type-Options: nosniff"); → Evita que se detecten de manera incorrecta los tipos MIME

header("Referrer-Policy: strict-origin-when-cross-origin"); → Controla cómo se manda el referer

header("Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'");
→ Evita que se usen recursos de fuera

Con estas cabeceras, el navegador tiene que usar políticas de seguridad más estrictas, lo que reduce el riesgo de ataques que manipulen el contenido o los recursos externos.

6. COMPONENTES VULNERABLES Y OBSOLETOS

6.1. DESCONOCIMIENTO DE VERSIONES EXACTAS

En un primer momento, la aplicación utilizaba imágenes antiguas de PHP, Apache y MariaDB dentro de su entorno Docker. Estas versiones presentaban riesgo de contener fallos de seguridad ya documentados.

Para arreglar este problema, se han actualizado las versiones de todas las imágenes a las más recientes y estables. En el caso de MariaDB, no se ha tomado la más reciente por cuestiones de compatibilidad; sin embargo, se ha optado por una versión más moderna y con soporte hasta 2029. Igualmente, se ha añadido soporte para conexiones cifradas utilizando SSL, lo que garantiza la comunicación entre los contenedores y el navegador.

Con estas mejoras, el entorno de ejecución queda protegido frente a vulnerabilidades conocidas en versiones anteriores y se garantiza un funcionamiento más seguro y fiable del sistema.

6.2. USO DE SOFTWARE OBSOLETO O SIN SOPORTE

Se detectó que el servidor Apache no contaba con una configuración optimizada de sus módulos, lo que suponía un riesgo de seguridad al mantener activadas extensiones innecesarias o no utilizadas. Este tipo de configuración puede ampliar la superficie de ataque del servidor, exponiendo componentes que podrían contener vulnerabilidades o permitir accesos indebidos.

La solución aplicada ha consistido en revisar y ajustar la configuración de Apache, habilitando únicamente los módulos estrictamente necesarios para el correcto funcionamiento de la aplicación. De esta forma se redujo la carga del servidor, se eliminó la exposición de servicios prescindibles y se mejoró la seguridad general del entorno.

Esta corrección contribuye a minimizar el riesgo de explotación de vulnerabilidades asociadas a módulos obsoletos o innecesarios, garantizando que el servidor utilice sólo los componentes requeridos por el proyecto.

6.3. DEPENDENCIAS EXTERNAS NO CONTROLADAS

Se observó que la aplicación no incluía políticas HTTP que limitaran la ejecución de contenido externo o la manipulación del sitio desde otras páginas.

Para resolverlo, se han implementado diferentes cabeceras de seguridad que restringen el acceso de la aplicación a recursos ajenos, impiden que otras páginas puedan incrustar su contenido y obligan al navegador a tratar todos los archivos según su tipo real.

Gracias a estas medidas, se evita el riesgo de ataques como el clickjacking o la inyección de scripts externos, mejorando la protección de los usuarios y de la propia aplicación.

Cabe añadir, que la librería con licencia libre fontawesome que se utilizaba anteriormente, ha sido descargada para así no tener que obtenerla de manera externa. Además, de esta manera, los cambios realizados en la misma no afectarían al servicio web.

7. FALLOS DE IDENTIFICACIÓN Y AUTENTICACIÓN

7.1. AUSENCIA DE AUTENTICACIÓN

Como ya se comentó en la auditoría, es una buena práctica implementar un segundo factor de autenticación. Debido a las limitaciones que tenemos, pues trabajamos en un servidor local no se ha podido implementar.

Aunque se pretendía enviar un código que fuese válido durante un periodo de tiempo al teléfono del usuario, el cual sería necesario para completar el acceso. Esta solución reduciría el riesgo de acceso no autorizado incluso en caso de que las credenciales fueran comprometidas.

7.2. FALTA DE LIMITACIÓN DE INTENTOS Y MENSAJES

En la versión anterior, no existía ningún mecanismo para limitar los intentos fallidos de iniciar sesión. Para la resolución de este problema, se ha implementado la limitación de intentos de inicio de sesión ya expuesta en el punto 4.1.

8. FALLOS EN LA INTEGRIDAD DE DATOS Y SOFTWARE

8.1. INCORPORACIÓN DE RECURSOS EXTERNOS SIN VERIFICAR SU INTEGRIDAD

La solución que se ha tomado es la coincide con la expuesta en el punto 6.3.

8.2. FALTA DE VERIFICACIÓN DE INTEGRIDAD DEL SOFTWARE LOCAL

Para los archivos locales del proyecto, hemos puesto en marcha un sistema de verificación de integridad que permite identificar cambios no permitidos en los recursos del sistema.

Primero, elaboramos el script `checksum.sh`, que tiene la función de producir el archivo `checksum.txt`. Este archivo incluye los hashes SHA256 de todos los archivos con extensiones `.js` y `.css`.

Después, desarrollamos el script `verificar_integridad.sh`, que compara los hashes actuales con los guardados en `checksum.txt` y almacena lo que se obtiene de la verificación en `integridad.log`.

Además, hemos configurado una tarea programada, `cron` que realiza esta verificación diariamente y registra los resultados en `integridad.log`. De esta forma, se garantiza que los archivos del sistema no sean alterados sin autorización y que el software local mantenga su integridad.

9. FALLOS EN LA MONITORIZACIÓN DE LA SEGURIDAD

Para asegurarnos de que todas las advertencias y errores se registren correctamente y no se muestren al usuario final, hemos configurado un sistema de logging seguro en `config.php` y `logger.php`. Conservamos todos los errores, advertencias y excepciones en un archivo de registro que se encuentra por defecto en `private/logs/errores.log`. En caso de que nuestra aplicación no tenga permisos para crear o escribir en esta carpeta, empleamos la carpeta temporal del sistema (`/tmp`) como lugar alternativo (Como se ha mencionado en el punto 4.5.)

De este modo, impedimos que la información sensible se filtre al navegador y los errores siguen a nuestro alcance como equipo de administración. Si la carpeta de logs no existe, la creamos de forma automática y hacemos que PHP registre todos los errores (`ini_set('log_errors', 1)` y `error_reporting(E_ALL)`). Asimismo, nuestro logger asegura la seguridad y la confidencialidad al filtrar información sensible (como contraseñas o tokens) de los datos antes de que sean escritos en el archivo. Gracias a esta solución, podemos auditar la actividad de la aplicación y detectar potenciales.

10. BIBLIOGRAFÍA

Mikel Egaña Aranguren — Seguridad Web (2024). Seguridad en sistemas Web. Repositorio de Github. Recuperado el 24 de octubre de 2025, de https://github.com/mikel-egana-aranguren/EHU-SGSSI-01/tree/main/Seguridad_web

Wikipedia — Inyección SQL. (s. f.). *Inyección SQL*. Wikipedia, la enciclopedia libre. Recuperado el 28 de octubre de 2025, de https://es.wikipedia.org/wiki/Inyecci%C3%B3n_SQL

Wikipedia — Cross-site request forgery. (s. f.). *Cross-site request forgery*. Wikipedia, the free encyclopedia. Recuperado el 28 de octubre de 2025, de https://en.wikipedia.org/wiki/Cross-site_request_forgery

Stack Overflow — How to use mysqli connection with SSL? (2015). *How to use mysqli connection with SSL?* Stack Overflow. Recuperado el 28 de octubre de 2025, de <https://stackoverflow.com/questions/29084442/how-to-use-mysqli-connection-with-ssl>

Raúl, H. (26 de febrero de 2017). *¿En qué consisten los ataques Parameter Tampering y cómo pueden evitarse?* HenryRaul (blog). Recuperado el 28 de octubre de 2025, de <https://henryraul.wordpress.com/2017/02/26/en-que-consisten-los-ataques-parameter-tampering-y-como-pueden-evitarse/>

Wikipedia — HTTP Strict Transport Security. (s. f.). HTTP Strict Transport Security. Wikipedia, the free encyclopedia. Recuperado el 29 de octubre de 2025, de https://en.wikipedia.org/wiki/HTTP_Strict_Transport_Security

PHP Documentation — session_set_cookie_params. (s. f.). session_set_cookie_params — Set the session cookie parameters. PHP.net. Recuperado el 29 de octubre de 2025, de <https://www.php.net/manual/en/function.session-set-cookie-params.php>

Wikipedia — HTTP Strict Transport Security. (s. f.). HTTP Strict Transport Security. Wikipedia, the free encyclopedia. Recuperado el 29 de octubre de 2025, de https://en.wikipedia.org/wiki/HTTP_Strict_Transport_Security

Wikipedia — Content Security Policy. (s. f.). Content Security Policy. Wikipedia, the free encyclopedia. Recuperado el 29 de octubre de 2025, de https://en.wikipedia.org/wiki/Content_Security_Policy

Stack Overflow — How to use mysqli connection with SSL? (2015). How to use mysqli connection with SSL? Stack Overflow. Recuperado el 29 de octubre de 2025, de <https://stackoverflow.com/questions/29084442/how-to-use-mysqli-connection-with-ssl>

WebDev: Cookies. <https://web.dev/articles/samesite-cookies-explained?hl=es>

OWASP Foundation — OWASP Top 10.(2021). Las Diez Vulnerabilidades más Críticas en Aplicaciones Web. OWASP Foundation. Recuperado el 30 de octubre de 2025, de <https://owasp.org/Top10/es/>

Apache Software Foundation — Apache HTTP Server SSL/TLS Encryption. (s. f.). Configuring HTTPS with mod_ssl. Apache HTTP Server Documentation. Recuperado el 5 de noviembre de 2025, de https://httpd.apache.org/docs/current/ssl/ssl_howto.html

Docker Documentation — Run Apache with SSL/TLS in Docker. (s. f.). Configure HTTPS containers. Docker Docs. Recuperado el 5 de noviembre de 2025, de <https://docs.docker.com/compose/compose-file/compose-file-v3/#ports>

OpenSSL Foundation — OpenSSL Command-Line Utilities. (s. f.). Creating Self-Signed Certificates. OpenSSL.org. Recuperado el 5 de noviembre de 2025, de <https://www.openssl.org/docs/manmaster/man1/openssl-req.html>

Wikipedia — HTTPS. (s. f.). Hypertext Transfer Protocol Secure. Wikipedia, la enciclopedia libre. Recuperado el 5 de noviembre de 2025, de https://es.wikipedia.org/wiki/Protocolo_seguro_de_transferencia_de_hipertexto